System Description

Recently, the Vanderbilt Aerospace Design Laboratory has had issues with payload landing detection in their current University Student Launch Initiative Competition (USLI) rocket mainly due to problems with their instrumentation and timing-based software logic. The implications for this lie in possible premature mid-air payload parachute detachment, resulting in catastrophic mission failure. Thus, a more robust solution is required to replace the previous landing detection methodology, one which will dramatically increase mission success probability. For this task, I propose a half bridge strain gauge circuit attached onto one of the payload's aluminum landing legs. Computation and data acquisition will be completed using an Arduino R4 Uno microcontroller and EMBSGB200 strain gage amplifier. These components are chosen mainly due to their relatively high sampling frequency and time-proven mission reliability. This method will use the magnitude of deflection in the aluminum legs during landing to determine both when the payload has made contact with the ground (i.e. landing detection) and the magnitude of force experienced during landing shock (i.e. landing G's). In general, this novel system may serve as a more robust way to detect landing because it does not rely on the payload's faulty pre-embedded instrumentation and timing-based logic, opting instead for a direct measurement of payload-to-ground interaction with minimal compromisable logic.

System Components and Their Integration and Construction

System Overview

To address the recurring issues with landing detection in VADL's current USLI payload, this project introduces a mechanical-electrical solution centered around strain-based ground detection. By integrating a half-bridge strain gauge configuration directly onto the payload's aluminum landing legs, the system enables real-time detection of ground contact and quantification of landing forces. The sensor data is processed by an Arduino R4 Uno paired with an EMBSGB200 strain gauge amplifier, offering both high sampling rates and operational reliability. Also, instead of using the current payload, I will construct a 'prototype payload' which is manufactured completely in-house, and uses none of the components that will fly on the current payload. That is to say, I will create a brand new payload to contribute to the merit of this project.

1) Leg Deployment Mechanism (LDM)

2) Parachute Detachment Mechanism (PDM)

3) Electronics Bay

4) Battery Bay

5) STEMnaut Crew Module (SCM)

6) Antenna Extension Mechanism (AEM)

7) RF System Antenna

8) Aft Bulk Plate

9) Fore Bulk Plate

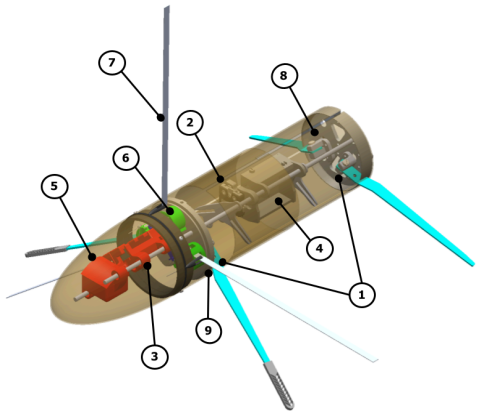

Figure 1. Current VADL payload with no-strain gage measurement device. Here, strain gages will be placed on at least one aluminum landing leg to measure deflection when the payload makes contact with the ground. The payload measures 20" from the tip of nose cone to aft bulk plate (8).

The Software Function and Actuation

The Arduino R4 Uno microcontroller reads the difference in voltage between two voltage dividers (a half bridge circuit) and subsequently computes strain, force applied to the end of the leg, and maximum landing G's [the maximum force recorded during landing shock divided by the resting weight of the payload (5.5 lb)]. If the maximum G's during a landing exceeds a threshold (1.2 G's) the software will both display the maximum G's felt during landing and actuate a servo arm to deploy an RF antenna. Although on the real payload, this antenna is

capable of transmitting and receiving RF, the ‘prototype payload’ will not have this feature as federal licensing is required..

To code this, I used concepts that I learned in class, mainly in the lab portion. In this way, some of the basic but core software logic concepts like analogRead, if statements, and for loops are used extensively in my software system operation. Additionally, concepts discussed in lecture like strain gage/half bridge logic, servo motion programming, voltage dividers, and microcontroller PWM communication allowed me to utilize components I would not have been able to otherwise, each extremely vital to the success of my project (i.e. digital voltmeters, servos, and the EMBSGB200 amplifier).

Figure 2. Half bridge strain gages imposed on prototype payload. Picture taken during field testing.

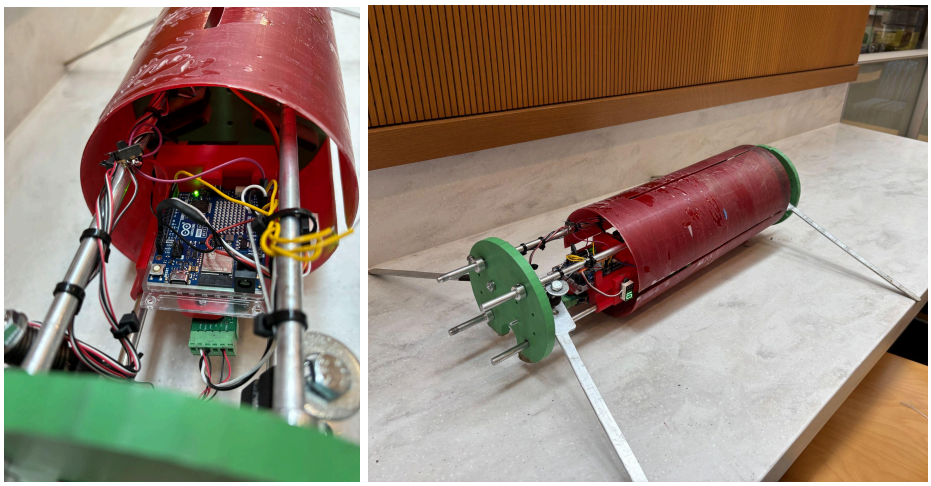

Figure 3. Completed ‘prototype payload’. The left image displays the payload’s electronics featuring the Arduino R4 Uno microcontroller, EMBSGB200 strain gauge amplifier, LCD voltmeter, and servo input terminals. The right image is the full payload assembly with a fiberglass frame, green ABS bulk plates, and red ABS electronics bay. Here, the LCD displays 1G, showing that the strain gages accurately record resting weight.

New Elements in System:

Component:	Role:	Justification:
6061 Aluminum Landing Leg	Provide support during payload landing.	Al 6061 is used *because* of its lower young's modulus relative to our other alternative, Al 7075. In this way, deformation can occur quicker, providing damping to the payload during landing.
Half Bridge Strain Gage Circuit	Provide method to quantify landing leg deflection.	Half bridge provides 1) higher thermal compensation 2) higher sensitivity 3) more common noise rejection relative to its quarter bridge counterpart.
Strain Gage Amplifier (EMBSGB200)	Amplifies microvolt-level output from the strain gauges to levels readable by the Arduino.	Used successfully during 2017 rocket flight, which implies the components in-flight reliability.
Arduino R4 Uno Microcontroller [4]	Provides analog to digital conversion at relatively high frequency and easily programmable.	Arduino's on-board ADC clock can reach speeds of 9500 Hz for this software configuration, which is enough to both 1) observe landing detection 2) record maximum Gs during landing.
LCD Voltmeter	Displays landing G's using PWM signal from Arduino.	As opposed to a LCD screen, the LCD voltmeter is a low power method to display maximum G's during landing.
MG90s Servo Motor	Flips RF antenna to 90 degrees for RF transmission.	This specific servo motor was found in the lab and used successfully in previous flights. Therefore, to replicate launch payload configurations, this model was used to activate the RF antenna.
RF Antenna	Transmits RF to NASA station.	Metallic strip used to transmit RF. Similar devices have been shown to effectively transmit RF up to one mile with current payload power configurations.
Fiberglass Airframe	Protect electronics bay during landing.	Used as airframe material in previous year's launch- proven as robust and high strength.
Aluminum Support Rods	Used to mount Electronics bay, bulk plates, and aluminum legs. These are the skeleton to the prototype payload.	Aluminum rods are lightweight and high strength to sustain direct loading applied during landing shock.
Bulk Plates	The green ABS bulk plates hold the landing legs to the support rods and provide a circular base for the airframe to reside.	ABS is a lightweight and high strength material. Although aluminum is often used for bulk plate material, ABS is preferred in this case because it is easier to manufacture while simultaneously meeting the strength requirements for landing shock.
Electronics Bay	Mounts Arduino and EMBSGB200 amplifier.	An ABS electronics bay is necessary to support electronic hardware during landing.
9 Volt Battery	To power Arduino and attached electronics.	A 9 V was used as it both adequately powered the Arduino and related components, as well as having a small form factor.

Table 1. Component list for those used in 'prototype payload'. Each part's role and justification for involvement has been listed as well. Green highlights indicate that the component was not covered directly in class. Yellow highlights indicate that the component was convered in class but this project extends its application (i.e. a quarter bridge was covered in class, but this project uses a half-bridge). Red indicates that this component was covered extensively in class. Although new progression was made in the software relative to what we learned in class (which will be shown in the appendix), the Arduino *as a microcontroller* was covered extensively.

Appendix

Code

```
#include <Servo.h>
//Constants
const int strainGaugePin = A3;
const int outputPin = 6;          //PWM output for voltmeter
const float Gage_Factor = 2.2;
const float VE = 5;
const float X = 0.225;
const float t = 0.0031;
const float b = 0.0145;
const float E = 68900000000;

const float GsThreshold = 1.2;    //Trigger threshold for landing
const int servoPin = 9;

unsigned long sampleStartMicros = 0;
unsigned int sampleCount = 0;

Servo actuator;

bool landingDetected = false;
int postLandingCyclesRemaining = 0;
float peakGs = 0.0;

void setup() {
  Serial.begin(115200);
  actuator.attach(servoPin);
  actuator.write(0); //Initial position
  sampleStartMicros = micros();
}

void loop() {
  //If landing is complete, hold PWM at peak Gs voltage
  if (landingDetected && postLandingCyclesRemaining <= 0) {
    float clampedPeak = constrain(peakGs, 0.0, 5.0); //Clamp to safe
voltage range
    int peakPWM = (int)(clampedPeak / 5.0 * 255.0);   //Convert to PWM
level
    analogWrite(outputPin, peakPWM);            //Hold value on voltmeter
```

```
    Serial.print("Landing complete. Max Gs: ");
    Serial.println(peakGs);
    while (true); //Halt further updates
  }

  //Read and process analog input
  int analogValue = analogRead(strainGaugePin);
  float voltage = (analogValue / 1023.0) * 5.0;
  float microstrain = abs((2.5 - voltage) * 2 / (VE * Gage_Factor)) * 1e3
* 4.2 * 1.033;
  float force = 2*microstrain * b * t * t * E / (6 * X) / 1e6;
  float Gs = force / 7.225;

  //Track peak Gs
  if (Gs > peakGs) {
    peakGs = Gs;
  }

  //Detect landing threshold crossing
  if (!landingDetected && Gs > GsThreshold) {
    landingDetected = true;
    postLandingCyclesRemaining = 10;  //Let 10 more frequency cycles run
  }

  //Actuate servo based on threshold
  if (peakGs > GsThreshold) {
    actuator.write(70);
  } else {
    actuator.write(0);
  }

  //Live PWM output for voltmeter
  float clampedGs = constrain(Gs, 0.0, 5.0);
  float pwmValue = (float)(clampedGs / 5.0 * 255.0)+.1;
  analogWrite(outputPin, pwmValue);

  //Sampling loop frequency counter
  sampleCount++;
  if (sampleCount >= 100) {
```

```
    unsigned long now = micros();
    unsigned long elapsed = now - sampleStartMicros;
    if (elapsed > 0) {
      float freq = (1000000.0 * sampleCount) / elapsed;
      Serial.println(freq);  //Output estimated frequency
    }

    sampleCount = 0;
    sampleStartMicros = micros();

    //Decrement post-landing countdown if triggered
    if (landingDetected && postLandingCyclesRemaining > 0) {
      postLandingCyclesRemaining--;
    }
  }
}
```

The above is the working code for prototype payload. To explain the functionality, first, the code initializes variables, either constants that are helpful for calculations, or pin numbers. The 'GsThreshold' is the G force at which landing is considered to be detected, as explained earlier, a threshold of 1.2 means that if at any point during landing, the strain gage measures that the payload is at least 1.2 times the resting weight of the payload, landing will be detected. Next, variables like sampleStartMicros and peakGs are assigned. In general, these track sample rate, manage landing detection, store the peak G force value, and control exactly how long the system computes for after landing. In the setup loop, the serial monitor is started, and the servo is attached to pin 9 with its initial angle set to 0°. Also, the sample timer is initialized here. In the main loop a couple of important tasks are initialized and iterated.

First, the output is freezed after landing. Meaning that if landining is detected and the post-countdown landing is over, then 'peakGs' is clamped to 5V (this is to ensure that is does not exceed the PWM limit of the Arduino is the landing Gs happens to be above 5 Gs). Next, we convert it to a PWM output and send it to the output pin.

In the next step, we read the analog value from the strain gage amplifier (pin A3) and convert the voltage signal to microstrain, force, and finally Gs.

Next, we track peak Gs, updating the value if the current G is larger than the maximum recorded G for that iteration.

Landing detection utilizes the if statement that if Gs exceed the threshold value (=1.2) and if landing was not already detected, we define it as a landing and keep running for 10 more cycles to see if there is another peak during that time.

If the peak Gs exceed the threshold (i.e. if landing is detected), the servo motor will actuate to 70° from the 0° it was initialized to. This flips up the RF antenna in a position where it is ready for transmission.

Next, we output the live G force as a PWM voltage to the voltmeter for visual logging. Also, we set the peak Gs to be output to the LCD voltmeter when the 10 cycle post-landing detection is complete. In this way, once the payload is dropped, the peak G value can be easily read from the LCD voltmeter. This allows for easy data recording during testing.

Finally, for frequency monitoring every 100 samples, the output frequency is calculated and printed to the serial monitor for user viewing. This is done to ensure the fast sample rate of the landing detection software, as high velocity landing has complicated collision mechanics that usually results in an extremely quick landing impulse.

Payload Manufacturing

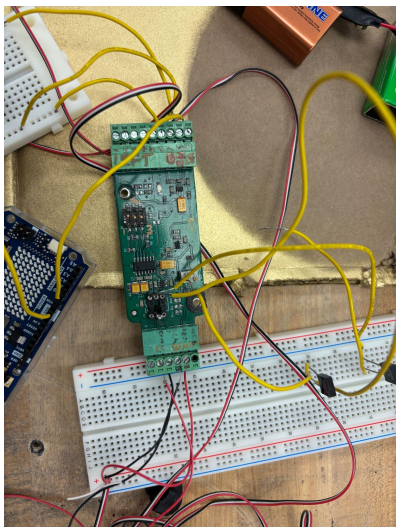

Figure 4. EMBSGB200 strain gage amplifier with strain gage wires as input. The amplified output is fed back into the Arduino microcontroller (pin A0) through a wire exiting the amplifier (terminal 8). Additionally, the resistors used for the half bridge are 120 ohm precision to ensure the accuracy of the half bridge circuit.

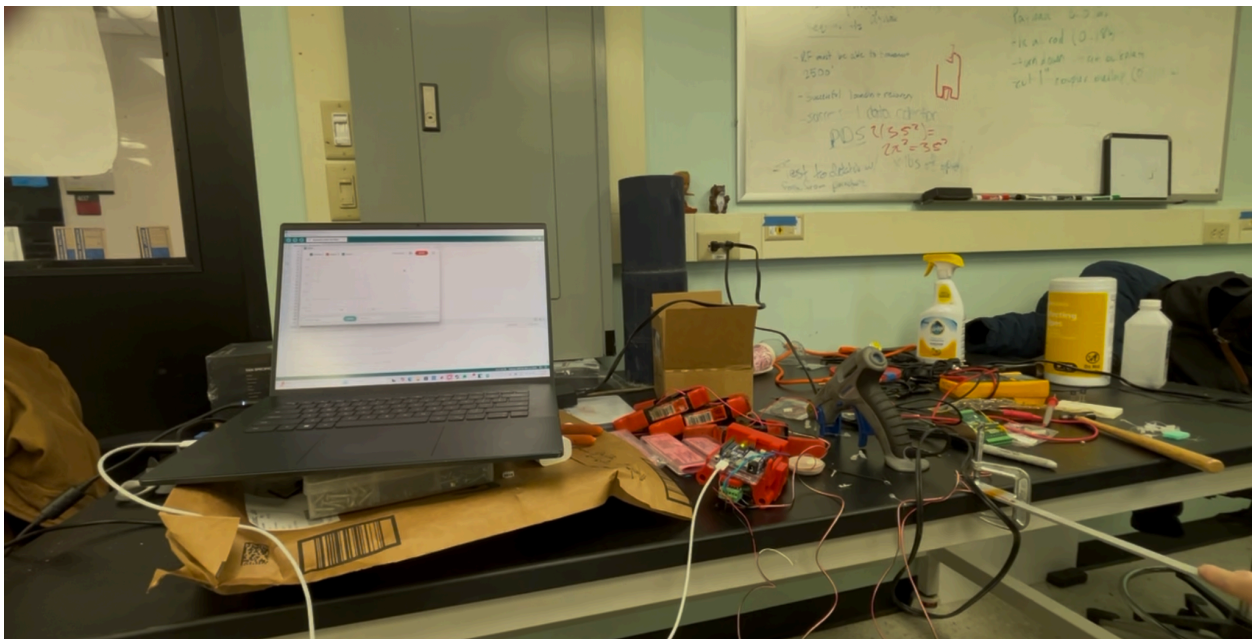

Figure 5. Testing of the red ABS electronics bay with arduino and EMBSGB200. The beam is purposely deflected with known weights for calibration. The deflection magnitude is output onto the serial monitor.

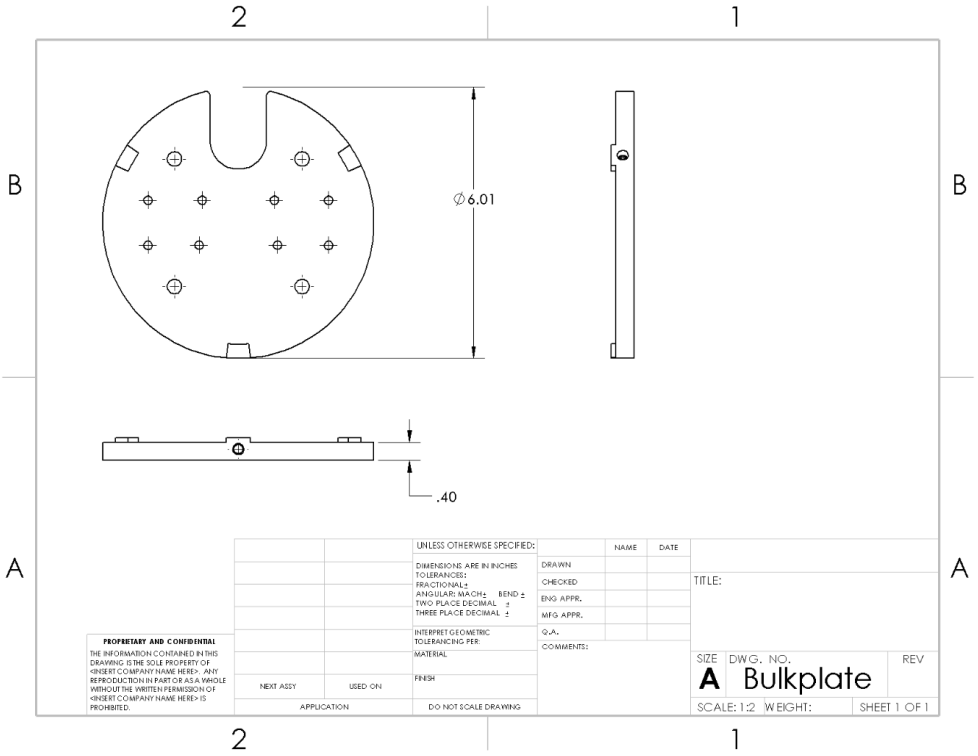

Figure 6. Drawing of green ABS bulkplates used in payload manufacturing.

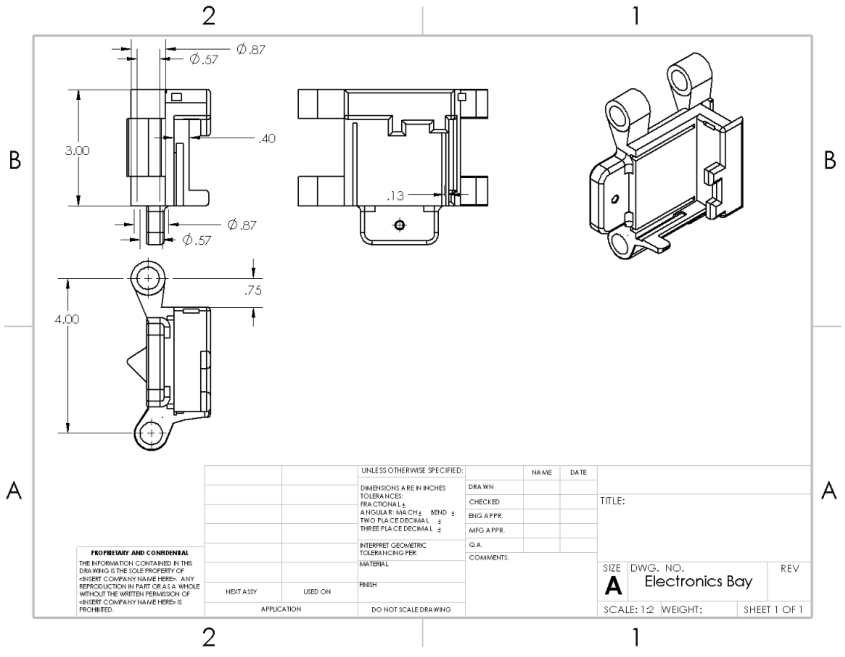

Figure 7. Drawing of electronics bay used to house payload electronics (Arduino, amplifier, and voltmeter).

X (m)		Imperial	Metric	Metrick			
0.225		Weight (lb)	Mass(kg)	Weight (N)	Calculated MicroStrain	Measured MicroStrain	Error
			0.168	1.64808	231.7402365	228	0.01640454593
t (m)			0.335	3.28635	462.1010668	456	0.0133795324
0.0031			0.5	4.905	689.7030847	676	0.02027083541
			0.666	6.53346	918.6845089	888	0.0345546271
b (m)							
0.0145							
E (Pa)							
68900000000							

$$\varepsilon = \frac{6X}{Ebt^2}F \quad \rightarrow \quad F = \frac{Ebt^2}{6X}\varepsilon$$

			Offset (V)	Mass (kg)	voltage (V)	microstrain	
			0.0005	0.5	-0.0595935	-673	

Figure 8. Calibration data used to inform the landing logic (i.e. force and G calculation) on the microcontroller.

Data and Experimental Setup

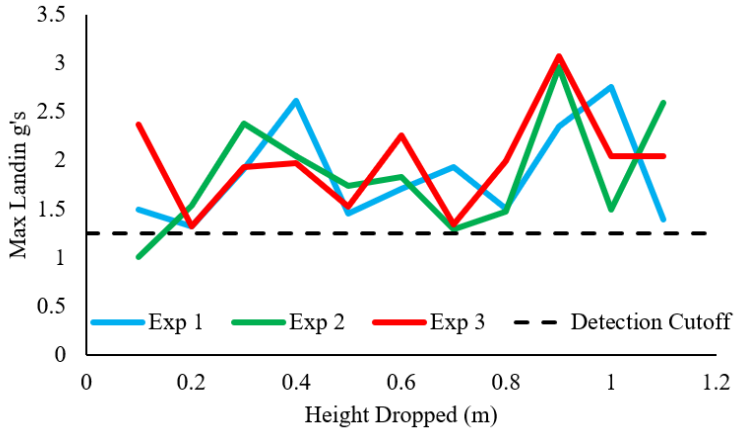

Figure 9. Study 1: Payload was aligned parallel to the ground and dropped. The purpose of this study was for control testing (i.e. a horizontal landing might aid in understanding landing dynamics). In this study, 3 experiments were performed. In each experiment, one drop was done at heights 10 cm above ground to 110 cm above ground. Three experiments implies that each drop height has 3 data points.

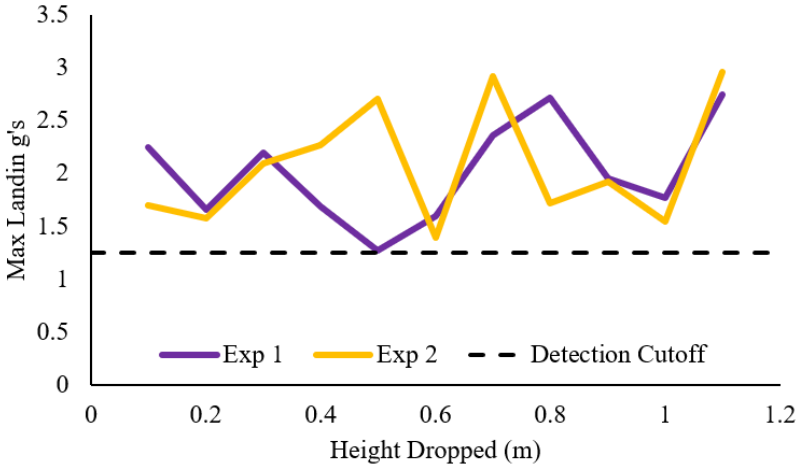

Figure 10. Study 2: Payload was aligned at a 30 degree angle to the ground and dropped. The purpose of this study is that it would replicate conditions closer to expected landing orientation during a VADL rocket launch.

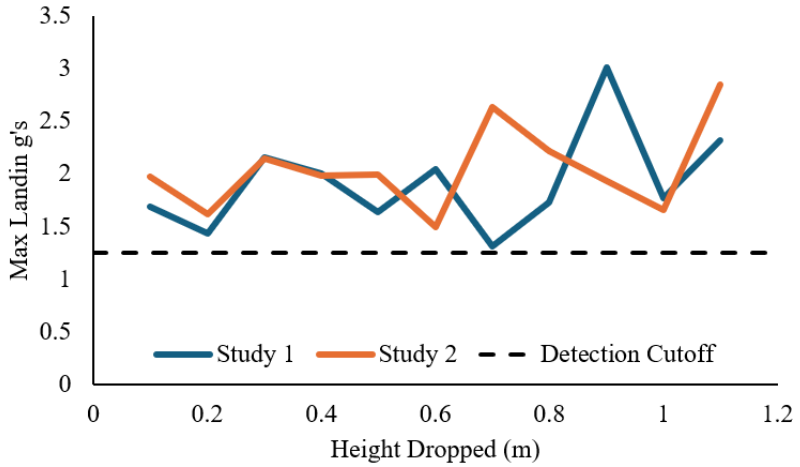

Figure 11. Averages of study 1 and study 2. Generally, this shows very little correlation between the average behavior of each study, inconsistent behavior for each drop height and inaccurate representation of force transferred to payload.

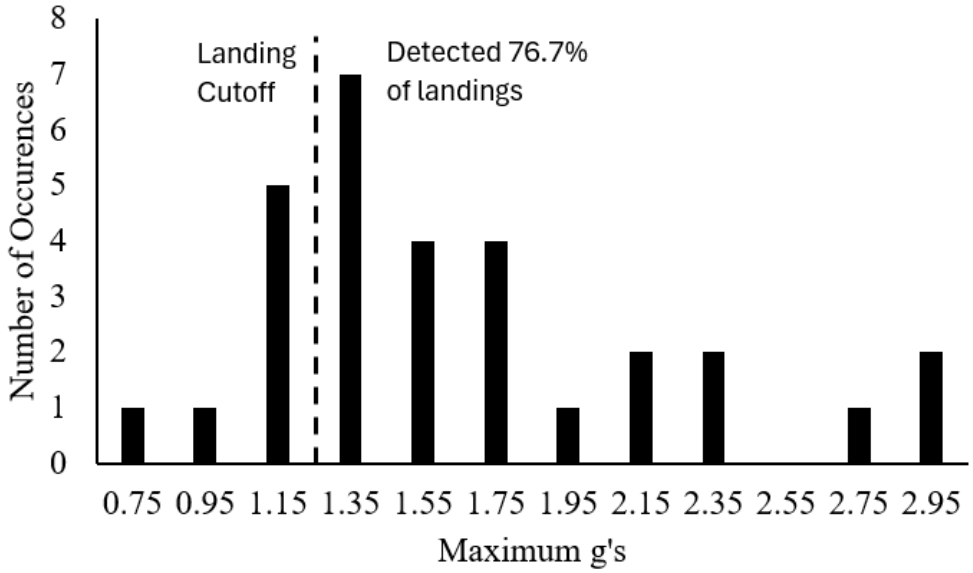

Figure 12. Study 3: Repeatedly dropped payload from 120 cm to test the precision of the system. In general, one should expect that a drop from a set height should produce the same maximum gs. Because this plot resembles a normal distribution ($\mu = 1.66$ and $\sigma = 0.66$) it admits that there is some variability in data. This is most likely caused by the relatively low sample rate of the first iteration of code (which was much less efficient than the code displayed in this document); one should expect a fairly linear relationship between drop height and maximum Gs.

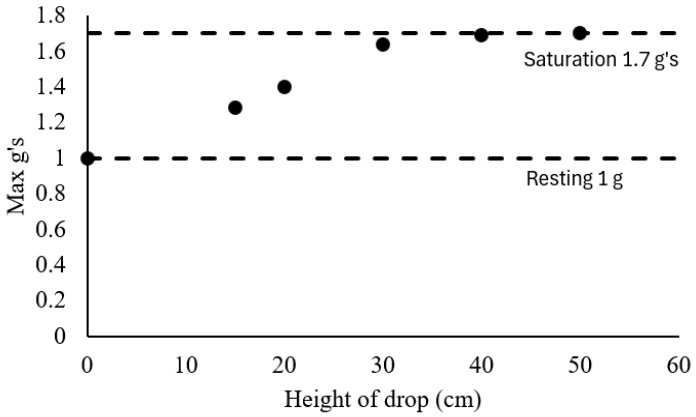

Figure 13. Max G's vs drop height relationship for increased frequency setup.Region between 0 cm and 40 cm suggests linear correlation between drop height and max g's. Saturation occurs because the microcontroller is only able to read 5V, and the lowest amplifier gain is 110, capping the strain gauge system at ~1.7 g's. In the linear region R^2 = 0.995 implying high correlation with the linear hypothesis. For each data point, 3 drops were performed and the average of maximum gs were recorded and plotted, similar to the methodology of the previous figures. Here, the sample rate was increased from 2kHz (from previous iteration) to about 9 kHz. The higher sample rate allows better collection of strain data as collision modelling requires extremely fast data acquisition frequency to model the quick deflection of the aluminum arm during drop. Additionally, increasing the sampling frequency of the board is an additional measure taken to increase the merit of this project, as this knowledge lies beyond the scope of the class.

Figure 14. Example drop of 20 cm above ground. The gray floor pad is implemented to simulate grass-dirt ground conditions, which is much softer than the lab's titled ground. Additionally, the 'g-spike' can be seen on the Arduino IDE serial plotter.

Figure 15. Outside experimental setup where previously shown drop test data was recorded. The ruler allows accurate placement of the payload to ensure standardized data acquisition.

Recommendation for Future Work

1. Use external ADC to sample at a higher rate/have larger voltage range.
    - 15kHz > and > +/- 10 V range
    - Increases saturation threshold
2. Perform precise drop height analysis & observe landing behavior (i.e. linear, quadratic, ect…).
3. Use higher resolution microcontroller
    - Pros: More reliable data
    - Cons: May be harder to program